

# OfferFlow HR — Beginner Guide (Short & Simple)

## Project overview

- What it does: Helps HR teams upload candidate CSVs, generate offer letters from templates, and send them by email in batches.
- Structure: Two main parts: backend/ (API + email sending) and frontend/ (React app UI).

## Quick run commands

- Start backend: `cd backend && npm install && npm run dev`
- Start frontend: `cd frontend && npm install && npm run dev`

## Backend files (what each file does, in very simple words)

- `backend/server.js`: Main server. Sets up API routes and starts the app on port 5000.
- `backend/package.json`: Lists backend dependencies and scripts (start, dev).
- `backend/.env`: Environment variables (e.g., DB URL) — keep secrets here.
- `backend/config/db.js`: Connects to MongoDB.
- `backend/config/seed.js`: Adds default companies and templates when DB is empty.
- `backend/config/constants.js`: Static constants (e.g., sample SVG logo).

## Models (data shapes stored in MongoDB)

- `backend/models/Company.js`: Company profile and SMTP settings.
- `backend/models/Template.js`: Email/template text and subject.
- `backend/models/Campaign.js`: One campaign groups candidates for a send.
- `backend/models/Candidate.js`: Each candidate's details and send-status.
- `backend/models/History.js`: Log of sends (sent/failed) for auditing.

## API routes (how frontend talks to backend)

- `backend/routes/companies.js`: (Company profiles) seed and manage company records.
- `backend/routes/templates.js`: CRUD for templates (list, create, update).
- `backend/routes/campaigns.js`: Manage campaigns and candidates (upload CSV, edit rows, start campaign).
- `backend/routes/email.js`: Test SMTP and send campaign emails (uses Nodemailer).
- `backend/routes/history.js`: Fetch send history logs.

## Important backend behaviors (short)

- Nodemailer is used to send emails from company SMTP settings.
- The send loop verifies SMTP first, then updates candidate status: Sending, Sent, Failed.
- Seed data uses `smtp.ethereal.email` by default — configure real SMTP in Settings to send real emails.

## Frontend files (what each file does, in very simple words)

- `frontend/package.json`: Frontend scripts (dev via Vite) and dependencies.
- `frontend/index.html`: HTML shell loaded by Vite.
- `frontend/src/main.jsx`: React entry point, mounts App.
- `frontend/src/App.jsx`: Main UI layout, chooses which page/component to show.
- `frontend/src/index.css`, `App.css`: Styling for the UI.
- `frontend/src/context/AppContext.jsx`: Central state and API helper functions (fetches data, triggers sends).

## Key components (what they do)

- `frontend/src/components/CsvUpload.jsx`: Drag-and-drop CSV upload and campaign creation.
- `frontend/src/components/DataReview.jsx`: Grid to view/edit candidate rows and select who to send to.
- `frontend/src/components/TemplateEditor.jsx`: Edit email subject/body and template tokens (`{{Name}}`, etc.).
- `frontend/src/components/PreviewSend.jsx`: Preview letters, set send options (delay, retries), and start sending; download PDFs.
- `frontend/src/components/SettingsTab.jsx`: Company settings and SMTP configuration; test SMTP here.
- `frontend/src/components/Sidebar.jsx`: Navigation sidebar.
- `frontend/src/components/Header.jsx`: Top header with tabs.
- `frontend/src/components/HistoryTab.jsx`: Shows past campaigns and allows downloading historical PDFs.

## How data flows (simple)

1. Create campaign (frontend -> `POST /api/campaigns`).

2. Upload CSV -> backend parses rows and saves Candidate documents linked to campaign.
3. Edit template -> save Template and link to Campaign.
4. Preview & Send -> frontend calls /api/email/send-campaign; backend verifies SMTP and sends emails.
5. History stored in History collection.

Common interview questions (short answers you can use)

- Q: How do templates insert candidate data?  
A: The code replaces tokens like {{Name}} with candidate fields before sending.
- Q: How does the app handle SMTP auth failures?  
A: It verifies credentials before sending and returns helpful suggestions (e.g., use Gmail App Passwords or Mailtrap).
- Q: How are CSV files validated?  
A: Frontend uses papaparse and enforces a strict column order; backend also validates email formats when saving.
- Q: Where is state managed in the frontend?  
A: AppContext.jsx (React Context) holds shared state and API functions.
- Q: How would you add email templates versions or previews?  
A: Add a version field to Template, create a template history collection, and expose a preview API.
- Q: How to improve deliverability?  
A: Use a transactional email provider (SendGrid/Mailgun/Postmark), set up SPF/DKIM, and respect rate limits.

Security & deployment notes (beginner)

- Never commit real SMTP passwords; store them in .env or secure secrets.
- For production, use process managers (PM2) and a real MongoDB (Atlas) instead of local MongoDB.
- Use HTTPS for the frontend and backend in production.

Files summary (plain list)

(Full file list in the project root; same short descriptions above)

---

End of quick guide. Good luck! If you want, I can:

- Turn this into a PDF and save it to project root, or
- Expand any section into longer explanations or diagrams.